

Assignment

Subject: Compiler Design

Units: 1, 2 and 3

Student Name: Tanisha Saxena

Enrollment No.: 2303051240246

ASSIGNMENT 1

1. What is a Compiler? Design the Analysis and Synthesis Model of Compiler.

A **compiler** is a system software that translates a program written in a **high-level language** (like C, Java) into **machine language or object code** that the computer can execute.

Analysis Phase (Front End)

The analysis phase reads the source program and checks for errors.

Steps involved:

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Intermediate Code Generation

Synthesis Phase (Back End)

The synthesis phase generates optimized machine code.

Steps involved:

1. Code Optimization
2. Target Code Generation

Diagram

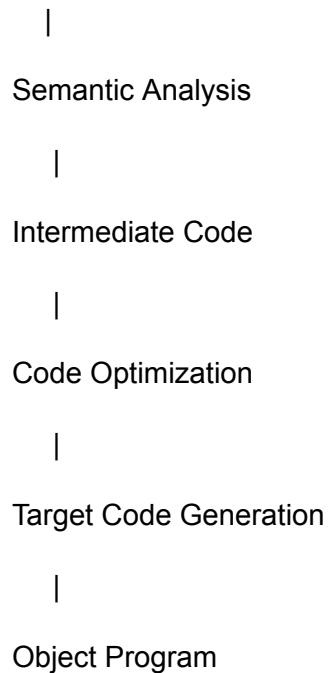
Source Program

|

Lexical Analysis

|

Syntax Analysis



2. Write five properties of a compiler

1. **Correctness** – The compiled program should give correct output.
 2. **Efficiency** – Generated code should use minimum memory and execute fast.
 3. **Error Detection** – Compiler should detect syntax and semantic errors.
 4. **Portability** – It should work on different machines with little modification.
 5. **Optimization** – It should improve the performance of generated code.
-

3. What is a Translator? Write steps to execute a program.

A **translator** is a program that converts a program written in one language into another language.

Examples:

- Compiler
- Interpreter
- Assembler

Steps to Execute a Program

1. Write the program in a high level language.
 2. Compile the program using a compiler.
 3. Fix errors if present.
 4. Generate object code.
 5. Link required libraries.
 6. Execute the program to get output.
-

4. Phases of Compiler (with diagram)

The compiler works in several phases.

Phases

1. **Lexical Analysis**
 - Converts characters into tokens.
2. **Syntax Analysis**
 - Checks grammatical structure.
3. **Semantic Analysis**
 - Checks meaning of statements.
4. **Intermediate Code Generation**
 - Produces intermediate representation.
5. **Code Optimization**
 - Improves efficiency.
6. **Target Code Generation**
 - Produces machine code.

Diagram

Source Code

|

Lexical Analyzer

|

Syntax Analyzer

|

Semantic Analyzer

|

Intermediate Code Generator

|

Code Optimizer

|

Target Code Generator

|

Machine Code

5. Short Notes

a) YACC

YACC (Yet Another Compiler Compiler) is a tool used to generate a parser for a programming language.

It works with **LEX** and uses context free grammar to produce syntax analyzers.

b) Pass

A **pass** is one complete processing of the source program by the compiler.

Types:

- Single Pass Compiler
 - Multi Pass Compiler
-

c) Bootstrapping

Bootstrapping is the technique of writing a compiler in the **same language that it compiles**.

Example:

A C compiler written in C language.

d) LEX Compiler

LEX is a tool used to generate **lexical analyzers**.

It converts input characters into tokens.

Example tokens:

- keywords
 - identifiers
 - operators
 - constants
-

e) Tokens, Patterns and Lexemes

Token:

Basic unit of a program.

Example:

`int x = 10;`

Tokens → `int`, `x`, `=`, `10`, `;`

Pattern:

Rule that describes token.

Example:

identifier → `letter(letter|digit)*`

Lexeme:

Actual text in program matching pattern.

Example:

`x`, `sum`, `total`

6. Steps to convert NFA to DFA

1. Start with the ϵ -closure of the start state.
2. Create a transition table.

3. For each input symbol find reachable states.
 4. Form new DFA states from the combinations.
 5. Repeat until no new states appear.
 6. Mark states containing final states as **final states**.
-

9. Regular Expression

A **regular expression** is a sequence of characters used to define patterns of strings.

a) Union Operation

$$R = R1 + R2$$

It represents strings in either R1 or R2.

Example

a + b

b) Concatenation

$$R = R1R2$$

Example

ab

c) Kleene Closure

$$R^*$$

Example

a*

Represents

ϵ , a, aa, aaa ...

d) Positive Closure

R^+

Example

a^+

Represents

$a, aa, aaa \dots$

e) Regular expression for strings ending with abb

Alphabet $\Sigma = \{a,b\}$

$(a+b)^*abb$

f) Regular expression for strings with any number of a's and b's except null string

$(a+b)^+$

ASSIGNMENT 2

1. Explain the parsing techniques with a hierarchical diagram.

Answer:

Parsing techniques are divided into Top-Down Parsing and Bottom-Up Parsing.

1. Top-Down Parsing:

- Starts from the start symbol and tries to derive the input string.
- Works by expanding non-terminals.
- Examples: Recursive Descent Parser, Predictive Parser (LL(1)).

2. Bottom-Up Parsing:

- Starts from the input string and reduces it to the start symbol by reversing rightmost derivation.

- Examples: Shift-Reduce Parser, LR, SLR, LALR, and Canonical LR parsers.

Hierarchical Diagram:

2. What are the problems associated with Top Down Parsing?

Answer:

- Left Recursion: Causes infinite recursion in parsers.
- Backtracking: Can make parsing inefficient.
- Ambiguity: Grammar may have multiple parse trees.
- Non-determinism: Parser may not decide which rule to apply.
- Limited Grammar Support: Cannot handle all context-free grammars.

3. Write the production rules to eliminate the left recursion and left factoring problems.

Answer:

Left Recursion Removal:

For production $A \rightarrow A\alpha \mid \beta$,

Convert to: $A \rightarrow \beta A'$ and $A' \rightarrow \alpha A' \mid \epsilon$

Left Factoring:

If $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$,

Convert to: $A \rightarrow \alpha A'$ and $A' \rightarrow \beta_1 \mid \beta_2$

4. Consider the following Grammar:

$A \rightarrow Aad \mid Aa \mid a$

$B \rightarrow Be \mid b$

Remove left recursion.

Answer:

For $A \rightarrow Aad \mid Aa \mid a$

Left recursion exists $\rightarrow A \rightarrow Aa \mid \beta$

So,

$A \rightarrow aA'$

$A' \rightarrow ad \mid a \mid \epsilon$

For $B \rightarrow Be \mid b$

$B \rightarrow bB'$

$B' \rightarrow eB' \mid \epsilon$

5. Do left factoring in the following grammar:

$A \rightarrow aAb \mid aA \mid a$
 $B \rightarrow bB \mid b$

Answer:

For A:

$A \rightarrow aA'$

$A' \rightarrow Ab \mid A \mid \epsilon$

For B:

$B \rightarrow bB'$

$B' \rightarrow B \mid \epsilon$

6. Write a short note on:

a. Ambiguity (with example):

A grammar is ambiguous if more than one parse tree exists for the same string.

Example:

$E \rightarrow E + E \mid E * E \mid id$

This grammar is ambiguous because “id + id * id” can have two different parse trees.

b. Recursive Descent Parser:

- A set of recursive procedures for each non-terminal.
- Matches the input using backtracking or prediction.
- Easy to implement for small grammars.

c. Predictive LL(1) Parser (Working):

- Top-down parser using a predictive parsing table.
- Each step predicts which production to use based on FIRST and FOLLOW sets.
- Eliminates backtracking.

d. Handle Pruning:

- In bottom-up parsing, a handle represents the substring that matches the right-hand side of a production to be reduced.
- Handle pruning reduces the handle into a non-terminal gradually.

7. Write rules to construct FIRST Function and FOLLOW Function.

Answer:

Rules for FIRST(X):

- If X is terminal $\rightarrow \text{FIRST}(X) = \{X\}$.
- If $X \rightarrow \epsilon \rightarrow \text{FIRST}(X)$ includes ϵ .
- If $X \rightarrow Y_1Y_2Y_3\dots$, $\rightarrow$ add $\text{FIRST}(Y_1)$ excluding ϵ , and if Y_1 derives ϵ , continue to Y_2 .

Rules for FOLLOW(X):

- Place \$ (end marker) in FOLLOW(S), where S is the start symbol.
- If $A \rightarrow \alpha B \beta$, everything in FIRST(β) except ϵ is added to FOLLOW(B).
- If $A \rightarrow \alpha B$ or $A \rightarrow \alpha B \beta$ where FIRST(β) contains ϵ , add FOLLOW(A) to FOLLOW(B).

8. Consider Grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow (E) \mid \text{id}$

Construct FIRST and FOLLOW function.

Answer:

FIRST(F): { (, id }

FIRST(T): { (, id }

FIRST(E): { (, id }

FOLLOW(E): {), \$ }

FOLLOW(T): { +,), \$ }

FOLLOW(F): { *, +,), \$ }

9. Write the algorithm to create Predictive parsing table with the scanning of input string.

Answer:

Algorithm:

1. Compute FIRST and FOLLOW sets.
2. For each production $A \rightarrow \alpha$:
 - For each terminal a in FIRST(α), add $A \rightarrow \alpha$ in $M[A, a]$.
 - If $\epsilon \in \text{FIRST}(\alpha)$, for each b in FOLLOW(A), add $A \rightarrow \alpha$ in $M[A, b]$.
3. For each step, check the top of the stack and input symbol to decide whether to match, push, or pop.

10. Show the following Grammar:

$S \rightarrow AaAb \mid BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

Is LL(1) and parse the input string "ba".

Answer:

For LL(1):

FIRST(A) = { ϵ }

$\text{FIRST}(B) = \{ \epsilon \}$

Thus, $\text{FIRST}(S) = \{ a, b \} \rightarrow \text{disjoint} \rightarrow \text{LL}(1) \text{ grammar.}$

Parsing "ba":

$S \rightarrow BbBa$

$\rightarrow bBa$

$\rightarrow ba$ ✓

11. Consider the grammar:

$S \rightarrow L = R \mid a$

$L \rightarrow *R \mid \text{id}$

Perform Shift Reduce Parsing for input string "a = id".

Answer:

Steps:

1. Shift a
2. Reduce by $R \rightarrow a$
3. Shift =
4. Shift id
5. Reduce by $L \rightarrow \text{id}$
6. Reduce by $S \rightarrow L=R$ ✓

ASSIGNMENT 3

12. Write the properties of LR parser with its structure. Also explain the techniques of LR parser.

Answer:

Properties:

- Accepts a large class of CFGs.
- Detects syntax errors early.
- Works from left-to-right and produces rightmost derivation in reverse.

Structure:

1. Input buffer
2. Stack for states and symbols
3. Parsing table (ACTION & GOTO)
4. Output (parse tree or derivation)

Techniques:

- SLR (Simple LR) – simplest form.
- LALR (Look-Ahead LR) – more efficient, used by many compilers.
- Canonical LR (CLR) – most powerful, uses LR(1) items.

13. Write a short note on:

a. Augmented Grammar:

A grammar with a new start symbol S' added $\rightarrow S' \rightarrow S\$$.

Used to define LR items properly.

b. Kernel items:

All items other than $[S' \rightarrow .S]$ and items where the dot is not at the beginning.

c. Rules of closure and goto operation:

- Closure(I): Add any item $[A \rightarrow \alpha.B\beta]$ for each production $B \rightarrow \gamma$.
- Goto(I, X): Move the dot after symbol X and take closure of the result.

d. Rules to construct LR(0) items:

1. Start with $[S' \rightarrow .S]$.
2. Apply closure repeatedly.
3. Apply goto for symbols.
4. Form LR(0) item sets.

14. Consider the following grammar:

$S \rightarrow A \mid aC \mid B \mid bBa$

$A \rightarrow d$

$B \rightarrow d$

Compute closure and goto.

Answer:

$I_0 = \text{closure}([S' \rightarrow .S]) =$

$\{ S' \rightarrow .S, S \rightarrow .A, S \rightarrow .B, S \rightarrow .aC, S \rightarrow .bBa, A \rightarrow .d, B \rightarrow .d \}$

$\text{Goto}(I_0, A) = \{ S \rightarrow A. \}$

$\text{Goto}(I_0, B) = \{ S \rightarrow B. \}$

$\text{Goto}(I_0, a) = \{ S \rightarrow a.C, C \rightarrow .d \}$

15. Write rules to construct SLR parsing table.

Answer:

1. Construct LR(0) items and their closures.
2. Build canonical collection of LR(0) items.

3. For each state i:
 - Shift if $[A \rightarrow \alpha.a\beta]$ exists.
 - Reduce if $[A \rightarrow \alpha.]$ and $a \in \text{FOLLOW}(A)$.
 - Accept if $[S' \rightarrow S.]$ and $a = \$$.

16. Construct LR parsing table for grammar:

$S \rightarrow aAA$
 $A \rightarrow aA \mid b$

Answer:

Short structure:

- $I_0: [S' \rightarrow .S], [S \rightarrow .aAA], [A \rightarrow .aA], [A \rightarrow .b]$
 By computing closures and gotos, build ACTION/GOTO table:

State	a	b	A	S
0	s3	s4	2	1
1	acc	2	s3	s4
2	s3	s4	5	3
3	s3	s4	6	4
4	r2	r2	5	r1
5	r1	r1	6	r3
6	r3	r3		

17. Write rules to construct LR(1) items.

Answer:

1. Start with $[S' \rightarrow .S, \$]$.
2. Apply closure: for each item $[A \rightarrow \alpha.B\beta, a]$, add $[B \rightarrow .\gamma, b]$ for every b in $\text{FIRST}(\beta a)$.
3. Apply goto for each symbol X .
4. Repeat until no new items appear.

18. What is LALR parser? Construct set of LR(1) items for grammar:

$S \rightarrow CC$
 $C \rightarrow aC \mid d$

Answer:

LALR Parser:

- Merges LR(1) states having same LR(0) core.
- Reduces size of LR(1) parser while maintaining power.

LR(1) Items:

$I_0 = [S' \rightarrow .S, \$], [S \rightarrow .CC, \$], [C \rightarrow .aC, a/d], [C \rightarrow .d, a/d] \dots$

Continue closure and goto operations.

19. Show the following grammar:

$S \rightarrow Aa \mid Ba \mid BbBa$
 $A \rightarrow d$
 $B \rightarrow d$

Answer:

This grammar is unambiguous. It can be parsed by SLR(1) and LALR(1) parsers.

FIRST(S): { a, b, d }

FOLLOW(S): { \$, a, b }

20. Write the comparison among SLR Parser, LALR Parser, and Canonical LR Parser.

Feature	SLR	LALR	Canonical LR
Power	Weakest	Moderate	Strongest
Table Size	Small	Medium	Large
Conflict Handling	Few	Fewer	Minimal
Lookahead	Uses FOLLOW	Merged 1-lookahead	1-token
Practical Use	Educational	Real-world (YACC)	Rare due to size

21. Calculate FIRST and FOLLOW for grammar:

$S \rightarrow aBDh$
 $B \rightarrow cC$
 $C \rightarrow bB \mid \epsilon$
 $D \rightarrow EF$
 $E \rightarrow g \mid \epsilon$
 $F \rightarrow f \mid g$

Answer:

FIRST(B): { c }

FIRST(C): { b, ϵ }

FIRST(D): { g, f, ϵ }

FIRST(E): { g, ϵ }

FIRST(F): { f, g }

FIRST(S): { a }

FOLLOW(S): { \$ }

FOLLOW(B): { h, b, g, f }

FOLLOW(C): { h }

FOLLOW(D): { h }
FOLLOW(E): { f, g }
FOLLOW(F): { h }